# ENUMS

# What is Enum

- Enum, introduced in Java 5, is a special data type that consists of a set of pre-defined named values separated by commas. These named values are also known as elements or enumerators or enum instances.
- According to Java naming conventions, it is recommended that we name constant with all capital letters

- One thing to keep in mind is that, unlike classes, enumerations neither inherit other classes nor can get extended(i.e become superclass). We can also add variables, methods, and constructors to it. The main objective of an enum is to define our own data types(Enumerated Data Types).

# Declaration of Enum in Java

Enum declaration can be done outside a Class or inside a Class but not inside a Method.

1.Outside the class

Gender.java

```
1
2 public enum Gender {
3     MALE,
4     FEMALE;
5
6 }
7
```

EnumMain.java

```
1
2 public class EnumMain {
3
4     public static void main(String[] args) {
5     Gender gender=Gender.FEMALE;
6     System.out.println("the gender is"+gender);
7
8     }
9
10 }
11
```

# Declaration of Enum in Java

## 2.Inside the class

```
public class EnumMain {
    enum Color {
        RED,
        GREEN,
        BLUE;
    }

    public static void main(String[] args) {

Color c=Color.GREEN;
System.out.println("the color is  "+c);
    }

}
```

# Enums with IF statement

```
public class EnumMain {
    enum Color {
        RED,
        GREEN,
        BLUE;
    }

    public static void main(String[] args) {

Color c=Color.GREEN;
if(c==Color.RED) {
    System.out.println("Red");
}else if(c==Color.GREEN) {
    System.out.println("Green");
}else if(c==Color.BLUE) {
    System.out.println("Blue");
}else {
    System.out.println("not in the list");
}
    }

}
```

# Enums with Switch case

```
public class EnumMain {
    enum Color {
        RED,
        GREEN,
        BLUE;
    }

    public static void main(String[] args) {

Color c=Color.RED;
switch(c) {
case RED:
    System.out.println("Red");
    break;
case GREEN:
    System.out.println("Green");
    break;
case BLUE:
    System.out.println("Blue");
    break;
default:
    System.out.println("Not in our list");
    break;
}
```

# Fields, Constructors and Methods within Enum

```
public class EnumMain {
    enum Laptop {
        HP(300),
        THINKPAD(200),
        DELL(100),
        MACKBOOK;
        private int price;

        private Laptop(int price) {
            this.price = price;
        }

        private Laptop() {
        }

        public int getPrice() {
            return price;
        }

        public void setPrice(int price) {
            this.price = price;
        }


    }

    public static void main(String[] args) {

Laptop p=Laptop.MACKBOOK;
p.setPrice(1000);
System.out.println("laptop price  "+p.name()+p.getPrice());
```

# Fields, Constructors and Methods within Enum

```
public class EnumMain {
    enum Laptop {
        HP(300),
        THINKPAD(200),
        DELL(100),
        MACKBOOK(1000);
    int price;

         Laptop(int price) {
            this.price = price;
        }


        public int getPrice() {
            return price;
        }



    }

    public static void main(String[] args) {

Laptop p=Laptop.MACKBOOK;
p.HP.price=500;
System.out.println("laptop price  "+p.name()+p.HP.price);
```

Price for HP has been changed because it is not final

# Looping Enum

```
public class EnumMain {
    enum Laptop {
        HP(300),
        THINKPAD(200),
        DELL(100),
        MACKBOOK(1000);
    int price;

         Laptop(int price) {
            this.price = price;
        }


        public int getPrice() {
            return price;
        }




    }

    public static void main(String[] args) {

Laptop p=Laptop.MACKBOOK;

System.out.println("laptop price  "+p.name()+p.HP.price);
for(Laptop lp:p.values())
{
System.out.println(lp.name()+"  "+lp.getPrice());
}
```

```
HP  300
THINKPAD  200
DELL  100
MACKBOOK  1000
```

# Enum and Inheritance

- All enums implicitly extend **java.lang.Enum class**. As a class can only extend **one** parent in Java, an enum cannot extend anything else.
- **toString() method** is overridden in **java.lang.Enum class**, which returns enum constant name.
- enum can implement many interfaces.